

Ops Sentinel

The AI SRE for data platforms, built on Exasol

Alerts in. Triaged, investigated, routed incidents out. Nothing changes without a human.

github.com/Shivathanu/ops-sentinel · Apache 2.0

2 a.m.: a pipeline fails and the finance dashboard goes stale



Who has the problem

Platform, data-engineering and on-call SRE teams at companies that make decisions on data pipelines.

Nightly ETL into a warehouse, finance and executive dashboards, services that depend on fresh tables.



Alerts everywhere

Prometheus, Grafana, CI and cron all fire, and they repeat. Which one matters?



Investigation by hand

Read logs, query the warehouse, trace lineage, look for the last time it happened.



Stale numbers reach decision-makers

The morning dashboard is already wrong before anyone notices.



Risky fixes

Alone, at night, with admin credentials and no record of why.

Why it matters: trust, time, money and risk



Trust

Decisions are made on stale data, and confidence in the data platform erodes.



Time

Resolution time goes to investigating. Repeats are investigated from scratch, because the lessons live in someone's head.



Money

Runaway queries burn compute for hours before a person looks.



Risk

Letting a script or a model “just fix it” with production credentials makes outages worse.

Teams need speed and control at the same time.

An AI SRE team that investigates, routes and asks first



Understand

- Groups duplicate alerts into one incident
- Agents triage and investigate with read-only tools
- Explains impact and root cause, with evidence



Route

- Routes by team, severity, labels and source
- Slack, signed webhooks, plugins
- Updates the same message as the incident evolves



Act safely

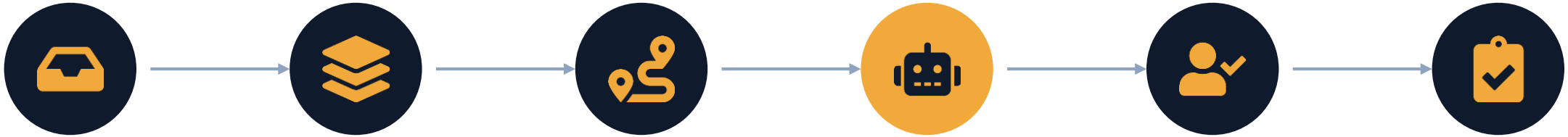
- Only pre-approved actions, never model-written code
- A named person approves
- Every step audited in Exasol

1 incident per failure, not N alerts

0 changes without a human's approval

100% of agent steps recorded for audit

How it works: from alert to audited outcome



1. Ingest

Any tool POSTs an alert

2. Correlate

Fingerprint: repeats join one incident

3. Route

Right channel by team and severity

4. Analyze

Triage, then investigation with tools

5. Approve

Pre-approved action, named approver

6. Audit

Every step stored in Exasol

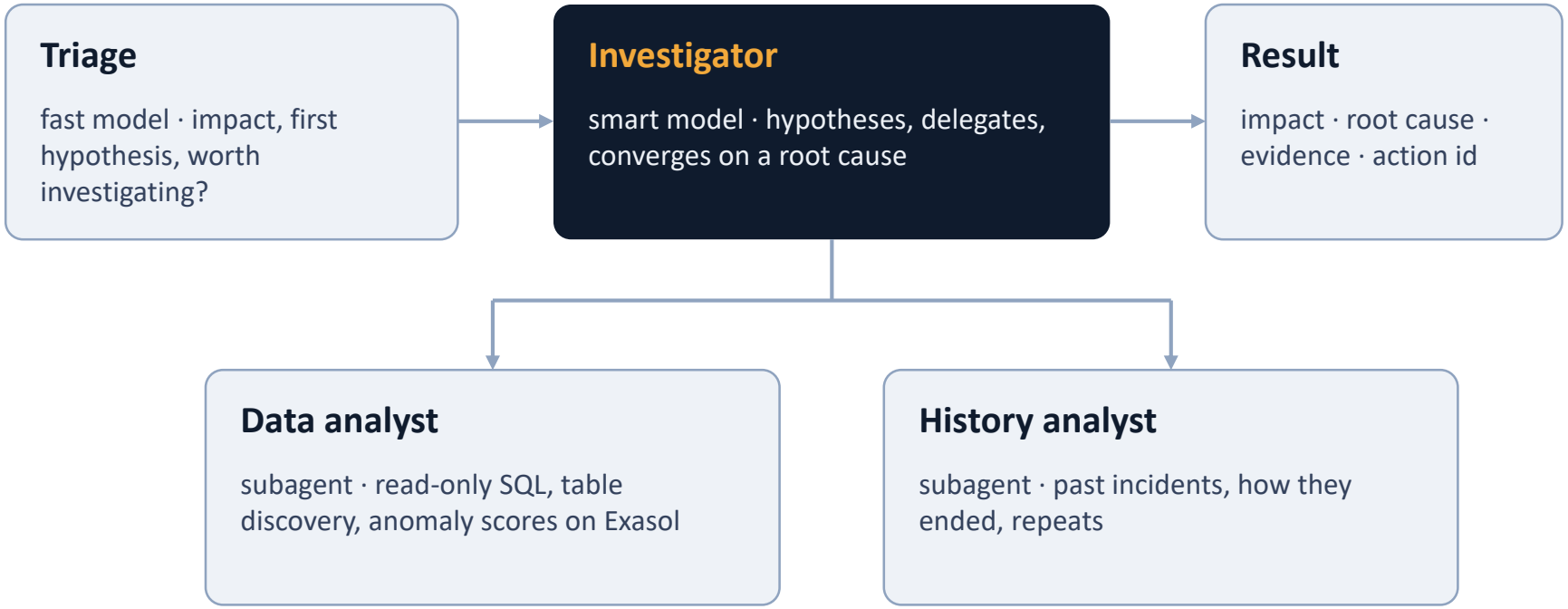
In

Prometheus Alertmanager, Grafana, CI and cron jobs, any JSON or text webhook, and the CLI runner on client machines.

Out

Slack messages updated in place, signed webhooks for any other system, and a REST API with each incident's full timeline.

AI-native: agents, subagents, tools and budgets



- Tool calling, arguments checked against each tool's schema
- Subagents work in their own context window
- Budgets per agent: steps, tokens, seconds, dollars
- Structured results, validated before they are used
- Claude, OpenAI or a local model through Ollama
- Teams override or add agents in sentinel.yaml

Toolsets incident history · action catalog · Exasol (read-only). Write tools never run from model output.

Every model call and tool call is recorded: AGENT_RUNS, AGENT_STEPS, EVIDENCE.

Exasol is the system of record and the investigation surface

Platform schema

Incidents, signals, timeline and every config version in OPS_SENTINEL.

Job queue

Leases, retries and dead letters in a table. No message broker to run.

Agent memory and audit

Runs, steps and full tool output as evidence, queryable with SQL.

Read-only investigation

Agents query pipelines, lineage and costs as mcp_readonly.

Python UDFs

FINGERPRINT, VECTOR_COSINE, ROBUST_ZSCORE: the logic runs next to the data.

Least privilege

Separate users to migrate, run, fix and investigate. The audit log is append-only.

Fast, but never unsupervised

Agents are read-only

Write tools are refused when a model calls them, and the attempt is recorded.

Model input is data

Alert text and tool output never count as instructions; secrets are redacted.

Pre-approved actions only

A fix is an action id from the team's catalog, never model-written code.

Human approval

Named approvers, signed Slack requests, one click runs a fix once.

Least-privilege users

The runtime user cannot rewrite the audit log or touch business tables.

Everything audited

Incidents, model calls, tool calls, notifications and decisions.

Every control fails closed: a missing key, token, approver or credential refuses the request.

From zero to an AI SRE in four commands

1

Bootstrap Exasol once

```
bootstrap_sentinel_roles.py --env-file .env
```

Schema and users; passwords go straight into .env

2

Describe the team

```
cp examples/sentinel.yaml .
```

Sources, channels, routes, models

3

Check it offline

```
sentinel-validate sentinel.yaml
```

Prints ingest URLs and where incidents go

4

Start it

```
docker compose up -d
```

api migrates the schema; workers start

One file per team. Secrets are references, so specs can live in Git.

sentinel.yaml

```
workspace: data-platform
sources:
  - { name: prometheus, type: alertmanager,
      token: "${env:AM_INGEST_TOKEN}" }
channels:
  oncall: { type: slack, channel: "#data-oncall",
            interactive: true }
routes:
  - match: { severity_at_least: high }
    notify: [oncall]
models:
  smart: { provider: anthropic,
           model: claude-opus-5-5 }
toolsets:
  exasol: { schemas: [SENTINEL] }
```

Demo: one failing pipeline, end to end

Alert	Alertmanager: pl_sales_clean_to_finance_bi failed 3 times in a row
Incident	One incident opens and is routed to the audit feed and to on-call in Slack
Triage	Impact in business terms: the Finance Executive BI dashboard is going stale
Investigation	The data analyst queries PIPELINE_LOGS: connection refused to warehouse wh01, after one slow run
Analysis	Root cause, evidence and the suggested pre-approved action, posted to Slack
Recovery	Repeats join the incident; a recovery resolves it; the full timeline is in Exasol

Where it stands today

Running on a live Exasol

- Ingest, correlation, routing
- Slack and signed webhook delivery
- Auto-resolve and full timeline
- Migrations, job queue, least-privilege users
- 224 automated tests

Built, live model run pending

- Agent runtime with subagents and budgets
- Read-only Exasol toolset
- Analysis attached to incidents and messages
- Claude, OpenAI and Ollama adapters

Next

- Policy-gated remediation for new incidents
- CLI runner on the new ingest API
- Knowledge and runbooks (RAG with VECTOR_COSINE)
- SSO, RBAC and a Helm chart

Roadmap



Where it goes: every team plugs its alerts, data and channels into one AI SRE that remembers what worked, proposes the fix, and runs it only with approval.



Trusted data. Faster recovery.

And no unsupervised changes.

Code: github.com/Shivathanu/ops-sentinel

Run it: docs/getting-started.md · Deploy it: docs/deployment.md

Licence: Apache 2.0